SC6103 Distributed Flight Information System

Course Project Report | 13 October 2026 | Protocol v1.0

Java server and Python clients over IPv4 UDP

Group member	Main responsibility	Actual contribution
Zhang Zhiyin	A: binary protocol, UDP server, invocation semantics and integration	33.33%
Peng Jinyu	B: flight business logic, monitor service, experiment coordination and report integration	33.33%
Ji Chengyu	C: Python client, retries, callback reception and console interface	33.33%

All three members contributed equally: one third each, displayed as 33.33% per member. The displayed sum is 99.99% solely because of rounding; the exact shares total 100%.

Classroom demonstration: Slot 11 | Saturday, 17 October 2026 | 14:15-14:30 (Singapore time) | LT8. All three members will attend; planned arrival is 14:00 for setup.

Abstract. This project implements six flight services using a manually encoded binary protocol and interoperable Java/Python programs. It supports bounded client retries, selectable at-least-once (ALO) and at-most-once (AMO) invocation, and time-limited server callbacks. The report explains the design and presents observed results from local automated verification and a three-computer experimental rehearsal on 12 October 2026. The LAN experiments include business validation, continuous callbacks, concurrent monitoring, flight-specific updates, and six controlled first-reply-loss runs. The measured outcomes demonstrate repeated non-idempotent effects under ALO and cached duplicate handling under AMO.

Implementation baseline: main commit `3241a333b38899ec65fe0ba8e8fcc12001b03967` (English console release).

Repository: https://github.com/Stevenhh7/SC6103-Distributed-System-Coursework

Contents

1. Objectives and architecture 3

2. Explicit binary protocol 4

3. Flight operations and business state 5

4. Retries and invocation semantics 9

5. Time-limited server callbacks 10

6. Fault experiments and observed results 15

7. Verification and experimental environment 21

8. Reproduction, conclusions and limitations 22

9. Evidence index and references 23

1. Objectives and architecture

The system provides flight information, reservations and time-limited seat monitoring over IPv4 UDP. It also implements an idempotent fare-setting operation and a non-idempotent fare-increase operation. A Java server interoperates with Python clients through an explicit binary protocol. The production system uses only the JDK and Python standard library; it does not use RMI, RPC frameworks, CORBA, Java object serialization or stream-based network marshalling.

The client menu and experiment driver call the same Invoker. The Invoker allocates a session/request identity, encodes the request once and uses one UDP socket for requests, replies and callbacks. The server receives a datagram, validates it, applies the selected invocation semantics and delegates business work to DefaultFlightService. That service shares one InMemoryMonitorService with the dispatcher. Business code returns typed responses and callback events rather than sending packets itself.

The server uses a sequential receive/dispatch loop. This makes history lookup, business execution and result recording ordered without a thread per request. A 250 ms receive timeout permits subscription cleanup while idle. The shutdown hook closes the socket to unblock reception. Business data, subscriptions and duplicate history are in memory and reset on restart.

Part A implements BinaryProtocolCodec, UdpTransport, UdpServer, RequestDispatcher, InMemoryRequestHistory, LossSimulator and structured ServerLog output. The shared business and client interfaces isolate protocol/transport work from flight-state changes and user interaction. Protocol errors retain sufficient local metadata to encode a safe error reply without changing the agreed wire format.

2. Explicit binary protocol

All multi-byte numbers use network byte order. Signed integers occupy four bytes; fares use IEEE 754 binary32. Locations and error text use strict UTF-8, prefixed by a four-byte byte count rather than a character count. Java manually shifts integer/UUID bytes and converts float bits; it does not serialize objects or use input/output streams to marshal messages. Python follows the same field layout. Dates contain five integers: year, month, day, hour and minute, interpreted in UTC+8.

Header field	Bytes	Meaning
version	1	Protocol version 1
messageType	1	Request 1, reply 2, callback 3
operation	2	Operations 1 through 6
sessionId	16	Nonzero UUID
requestId	4	Positive signed integer
status	2	Success 0 or defined error 1 through 9
semantics	1	ALO 1 or AMO 2
reserved	1	Must be zero
bodyLength	4	Exact bytes following the header

The header is exactly 32 bytes. The application maximum is 1024 bytes, while reception uses a 65535-byte buffer to detect oversized UDP messages rather than silently truncating them to the application limit. Decoding uses the actual packet offset and length. It rejects missing fields, extra bytes, negative lengths, malformed UTF-8, invalid response dates and non-finite response fares. Location text is limited to 128 encoded bytes and error text to 256 bytes. Typed but business-invalid request values are passed to business validation and receive a normal business error.

Malformed packets with a trustworthy request identity receive a protocol error; identities that cannot safely be recovered are silently dropped. Oversized messages return status 9 when identity is trustworthy. Unsupported operations return 6; mode mismatch returns 7 and echoes the requested mode so the client can match the error. Malformed-message status is 5 and conflicting reuse of an AMO request key is 8. These pre-business failures do not populate business history.

The repository's protocol specification v1.0 defines the normative byte layout, status definitions and fixed vectors. Java-to-Python and Python-to-Java vector tests cover requests, successful responses, errors and callbacks, including Chinese UTF-8 locations.

3. Flight operations and business state

Operation	Request body	Successful response body
QUERY_ROUTE (1)	Source and destination strings	Count followed by all matching flight IDs
QUERY_FLIGHT (2)	Flight ID	Five departure-time integers, fare, available seats
RESERVE_SEATS (3)	Flight ID, quantity	Flight ID, remaining seats
MONITOR (4)	Flight ID, duration in seconds	Flight ID, remaining milliseconds
SET_AIRFARE (5)	Flight ID, new fare	Flight ID, resulting fare
INCREASE_AIRFARE (6)	Flight ID, positive increment	Flight ID, resulting fare

Route results are sorted and include all matches. Detail responses deliberately do not repeat the flight ID: clients already have the request context, and both implementations follow the agreed response shape. Reservation validation completes before state changes; invalid quantities, unknown flights and insufficient seats produce errors with no seat or sequence-number side effects.

The six-flight seed includes two Singapore-to-Beijing matches, a sold-out Singapore-to-Shanghai flight, Chinese-location routes and a reverse route. Flight 1001 starts with 10 seats, fare 100 and update sequence 0. Each experiment starts a fresh server and session to restore this known state. The controlled reservation quantity is 1, SET fare is 120 and INCREASE delta is 20.

SET_AIRFARE is idempotent with respect to fare state: applying the same assignment twice leaves the same value. INCREASE_AIRFARE is non-idempotent: two successful additions of 20 change 100 to 140. Reserving seats is also non-idempotent. Business validation rejects non-finite, out-of-range and ineffective floating-point changes as required by the shared specification. Fare changes do not generate seat-change callbacks.

3.1 Observed business operations on the LAN

The query client returned both flight IDs for Singapore-to-Beijing and for the Chinese UTF-8 route Beijing-to-Shanghai. Flight 1001 initially showed 10 seats and fare 100 SGD. The reservation client returned status 1 for an unmatched route, status 2 for flight 9999, and status 3 both for sold-out flight 1003 and for requesting 11 seats on flight 1001. These are server business errors, not client-only input validation.

In the successful sequence, reserving one seat changed 1001 to 9 seats. SET 120 followed by INCREASE 20 produced fare 140 without further seat changes. After the monitor interval, a second successful reservation left 8 seats and fare 140. Figures 1-4 show the actual client screens. Screenshot 04 provides the corresponding server transaction sequence, including the initial callback and the later reservation with no callback to the expired registration in that captured transaction.

Figure 1. Route queries, UTF-8 data and initial flight details

```
PS C:\Users\Administrator> Set-Location 'D:\ntu\103\groupwork'
PS D:\ntu\103\groupwork> python -X utf8 -m client --server 10.91.127.7 --port 6789 --semantics amo --log-file evidence/d
emo/20261012-03-basic-en-01/client-B.jsonl
SC6103 Flight Client; mode=AMO; session=06aa4974-ca5b-43b0-b635-f4fabfb94be8

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 1
Source: Singapore
Destination: Beijing
Request 06aa4974-ca5b-43b0-b635-f4fabfb94be8:1; attempts: 1
Matching flights: 1001, 1002

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 1
Source: 北京
Destination: 上海
Request 06aa4974-ca5b-43b0-b635-f4fabfb94be8:2; attempts: 1
Matching flights: 1004, 1005

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 2
Flight ID: 1001
Request 06aa4974-ca5b-43b0-b635-f4fabfb94be8:3; attempts: 1
Departure 2026-10-17 09:30 UTC+8; airfare 100.00 SGD; available seats 10
```

Figure 1. B queried Singapore-to-Beijing (1001, 1002), queried Chinese UTF-8 locations (1004, 1005), and read flight 1001 as 10 seats and fare 100 SGD. Original supplied screenshot 24.

The UTF-8 route result checks variable-length string interoperability; the English console preserves the entered location data. Requests 1-3 belong to B session 06aa4974-ca5b-43b0-b635-f4fabfb94be8.

Figure 2. Server business errors returned to the client

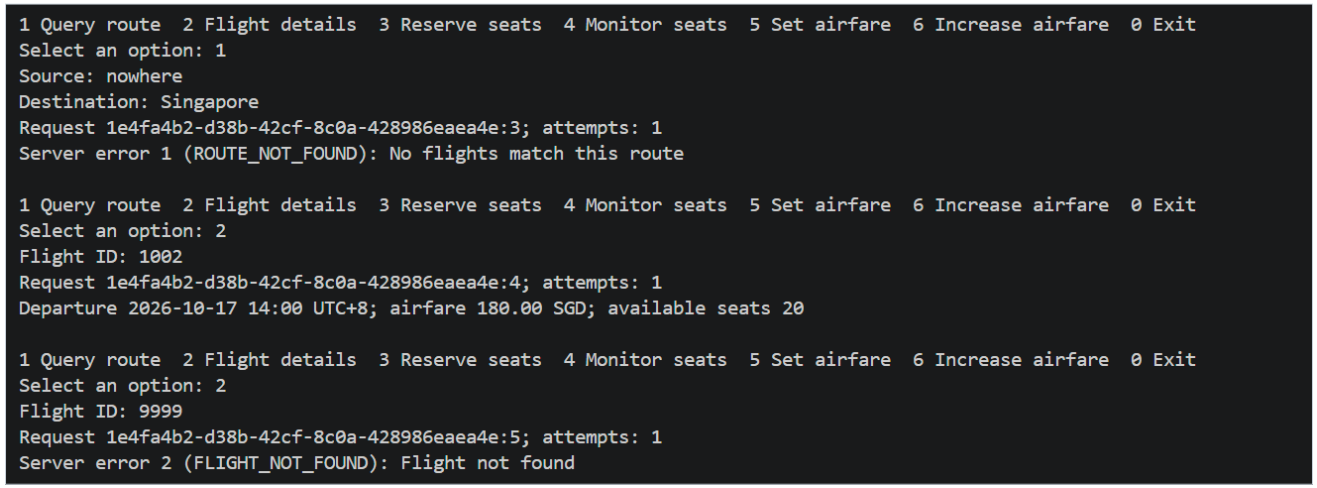


Figure 2a. C received ROUTE_NOT_FOUND (1), read flight 1002 successfully, and received FLIGHT_NOT_FOUND (2) for 9999. Original screenshot 01.

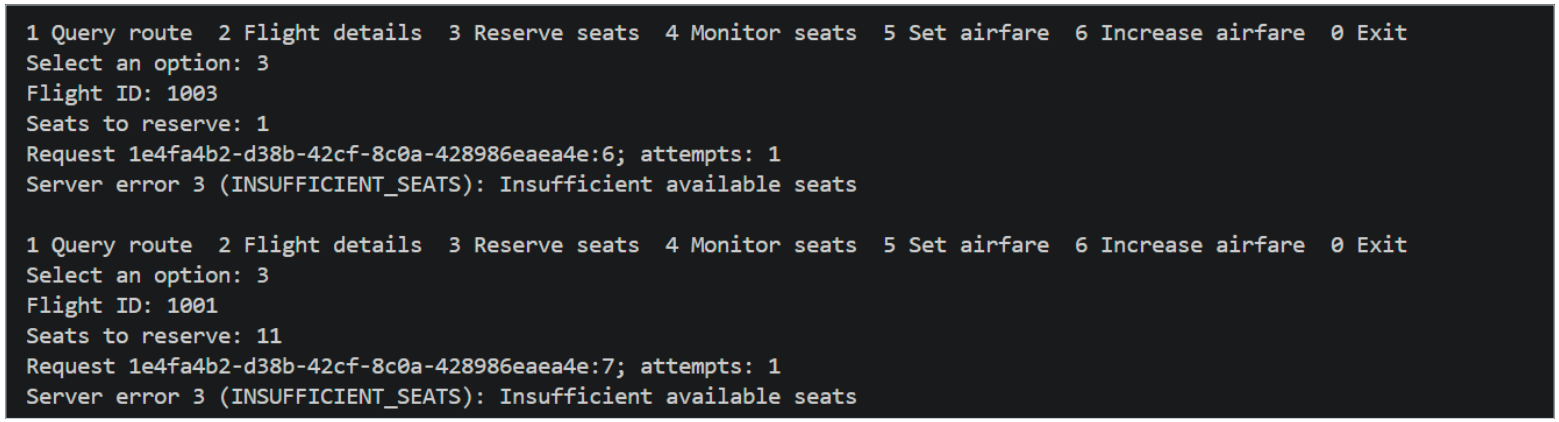


Figure 2b. Reservations on sold-out flight 1003 and for 11 seats on flight 1001 returned INSUFFICIENT_SEATS (3). Original screenshot 02.

Both panels use C session 1e4fa4b2-d38b-42cf-8c0a-428986eaea4e. Errors are explicit replies rather than silence or successful writes.

Figure 3. Idempotent SET and non-idempotent INCREASE

```
1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 5
Flight ID: 1001
New airfare (SGD): 120
Request 1e4fa4b2-d38b-42cf-8c0a-428986eaea4e:10; attempts: 1
Airfare updated: flight 1001, total airfare: 120.00 SGD

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 6
Flight ID: 1001
Amount to increase (SGD): 20
Request 1e4fa4b2-d38b-42cf-8c0a-428986eaea4e:11; attempts: 1
Airfare updated: flight 1001, total airfare: 140.00 SGD

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 2
Flight ID: 1001
Request 1e4fa4b2-d38b-42cf-8c0a-428986eaea4e:12; attempts: 1
Departure 2026-10-17 09:30 UTC+8; airfare 140.00 SGD; available seats 9
```

Figure 3. C set flight 1001 to 120 SGD, increased it by 20 SGD, and read 140 SGD with 9 seats. Original screenshot 03.

Requests 10 and 11 are operations 5 and 6; request 12 is a read-only confirmation. These fare operations do not change the available-seat count.

4. Retries and invocation semantics

A client creates a fresh UUID session and increasing positive request IDs. A logical call is encoded once. Every retry retains identical bytes, UUID, request ID and source socket. Defaults are a 1000 ms timeout and five total attempts, including the initial send. Irrelevant, malformed or late messages do not extend an attempt's absolute deadline. Replies must match the configured server endpoint and outstanding request identity, operation and semantics.

In ALO mode, the server executes every valid received request, including duplicates. Retries improve the chance of completion, but a finite retry budget does not guarantee delivery when all requests are lost. A lost reply may cause a non-idempotent operation to execute multiple times.

In AMO mode, the server first checks history keyed by (session UUID, request ID). An entry retains the original request bytes, peer endpoint and immutable result. Matching duplicates replay the result without invoking the business method again. Reuse with different bytes or a different peer is rejected. Successes and business errors are saved before ordinary reply transmission or simulated reply loss. Thus a retry after a lost business error receives the same error even if other clients subsequently change the state.

The ordering is: validate trustworthy header and mode; check AMO history; decode a new request; execute business logic; encode and save the result; drop or send the ordinary reply; attempt each eligible callback. History has no eviction during a server process lifetime. ALO bypasses duplicate suppression. Both modes use the same business implementation.

AMO is an at-most-once execution guarantee within the retained in-memory history, not an exactly-once or crash-recovery guarantee. Restart discards history. If attempts expire, the client reports that confirmation was not obtained and the execution result is unknown. It does not automatically switch to a new identity and repeat a reservation or fare increase. A separate query can inspect current state but is not a transaction receipt under concurrent clients.

5. Time-limited server callbacks

MONITOR registers a client for a flight for 1 to 3600 seconds. Multiple clients may monitor the same flight; monitoring one flight does not subscribe to another. Successful reservations create events containing flight ID, remaining seats and increasing update sequence. The dispatcher sends a 44-byte callback from the same server socket directly to each active subscriber. Callback identity is the original registration UUID/request ID, not the reservation identity.

An ordinary reply being lost or its send raising an I/O exception does not suppress the callbacks produced by that reservation. One callback recipient failing does not stop attempts to the others. The server checks subscription activity immediately before each send and periodically purges expired registrations during idle reception. AMO replay never emits a second copy of the business event.

A monitor confirmation is the sole dynamic replay exception. Immediately before initial or duplicate confirmation transmission, the dispatcher reads the original registration's remaining lifetime. It does not re-register or extend that subscription. If the registration expired or was replaced, remainingMillis is zero. ALO duplicates do re-execute registration, consistent with that mode.

The client begins its local monitoring deadline from a successful confirmation's remainingMillis. Duplicate confirmations do not renew it. Before confirmation, only the highest-sequence callback for the pending registration is retained; after confirmation, only strictly increasing sequences from the expected server and registration are shown. Gaps are allowed because callbacks have no acknowledgement or retransmission. The menu resumes when monitoring ends. These callbacks provide best-effort updates, not a reliable event stream.

5.1 Single monitor and natural expiry: observed evidence

B registered flight 1001 for 60 seconds using session 06aa4974-ca5b-43b0-b635-f4fabfb94be8, request 4. The confirmation reported 59,991 ms remaining. C's successful reservation request 8 triggered a callback with 9 seats and sequence 1; both identities appear in the server capture (original screenshot 04). B's retained JSON Lines show MONITOR_START at 8384.734 and MONITOR_END at 8444.750 on B's own monotonic clock: 60.016 seconds elapsed, with the end after the recorded deadline 8444.725. No cross-computer clocks are subtracted.

The monitor resumed the menu naturally. C's later reservation request 13 reduced seats to 8. Its captured server transaction contains the successful reply and no CALLBACK_SENT to B's expired registration before the subsequent query. B then queried 8 seats and fare 140 (Figure 4). This combines client lifetime evidence with the provided server transaction capture, rather than treating a returned menu alone as proof of server expiry.

5.2 Concurrent monitoring and flight-specific observations

For the concurrent test, B and C registered the same flight 1001 from different IP addresses. The writer was a separate process on C's computer. Two reservations changed seats 10 -> 9 -> 8; both registered clients displayed sequences 1 and 2. The server capture contains four corresponding CALLBACK_SENT events: two recipients for each reservation (Figures 5-6). This is three physical computers and four processes, not four physical computers.

In the isolation test, B monitored 1001 while C monitored 1002. The writer reserved one seat on each. B showed only 1001 at 9 seats, sequence 1; C showed only 1002 at 19 seats, sequence 1. B's JSON Lines contain no 1002 callback during its recorded interval. Figure 7 confirms client-side flight-specific observations. The original A server JSON Lines in evidence/demo/20261012-06-isolation-01/server.jsonl additionally contain exactly two CALLBACK_SENT events: flight 1001 / 9 seats / sequence 1 to B at 10.91.162.52:58119, and flight 1002 / 19 seats / sequence 1 to C at 10.91.33.41:53423. Their registration and writer identities match the client records. The full transaction log contains no cross-flight delivery.

Figure 4. Single callback, natural expiry and final state

```
1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 4
Flight ID: 1001
Monitor duration in seconds (1..3600): 60
Request 06aa4974-ca5b-43b0-b635-f4fabfb94be8:4; attempts: 1
Monitor confirmed: flight 1001, remaining 59991 ms; menu input is paused during monitoring.
Seat update: flight 1001, available seats 9, sequence 1
Monitoring ended for flight 1001.
```

Figure 4a. B registered 1001 for 60 seconds, received 9 seats with sequence 1, and returned naturally at expiry. Original screenshot 25.

```
1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 2
Flight ID: 1001
Request 06aa4974-ca5b-43b0-b635-f4fabfb94be8:5; attempts: 1
Departure 2026-10-17 09:30 UTC+8; airfare 140.00 SGD; available seats 8
```

Figure 4b. B's subsequent read-only request 5 showed fare 140 SGD and 8 seats. Original screenshot 26.

The retained B log and server screenshot 04 correlate this registration with C's first reservation (request 8) and post-interval reservation (request 13). The second reservation did not produce a captured callback to the expired registration.

Figure 5. Both registered clients receive both updates

```
PS D:\ntu\103\groupwork> python -X utf8 -m client --server 10.91.127.7 --port 6789 --semantics amo --log-file evidence/
emo/20261012-05-dual-en-01/client-B.jsonl
SC6103 Flight Client; mode=AMO; session=ebecbaba-b8de-47d0-bdba-0b93a75eed40

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 2
Flight ID: 1001
Request ebecbaba-b8de-47d0-bdba-0b93a75eed40:1; attempts: 1
Departure 2026-10-17 09:30 UTC+8; airfare 100.00 SGD; available seats 10

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 4
Flight ID: 1001
Monitor duration in seconds (1..3600): 60
Request ebecbaba-b8de-47d0-bdba-0b93a75eed40:2; attempts: 1
Monitor confirmed: flight 1001, remaining 59999 ms; menu input is paused during monitoring.
Seat update: flight 1001, available seats 9, sequence 1
Seat update: flight 1001, available seats 8, sequence 2
Monitoring ended for flight 1001.
```

Figure 5a. B began from 10 seats / 100 SGD and received 9 / sequence 1 followed by 8 / sequence 2. Original screenshot 07; screenshot 27 is an identical duplicate and is not printed again.

```
1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 4
Flight ID: 1001
Monitor duration in seconds (1..3600): 60
Request f019af15-cba3-41ed-b3b1-12136403df21:1; attempts: 1
Monitor confirmed: flight 1001, remaining 59990 ms; menu input is paused during monitoring.
Seat update: flight 1001, available seats 9, sequence 1
Seat update: flight 1001, available seats 8, sequence 2
Monitoring ended for flight 1001.
```

Figure 5b. C's independent registration received the same two updates and ended naturally. Original screenshot 05.

B registration: ebecbaba-b8de-47d0-bdba-0b93a75eed40:2. C registration: f019af15-cba3-41ed-b3b1-12136403df21:1. Writer session: 370abbf3-49ed-4429-a9a7-9ae33abd9223 (requests 1 and 2).

Figure 6. Server callback recipients for the concurrent test

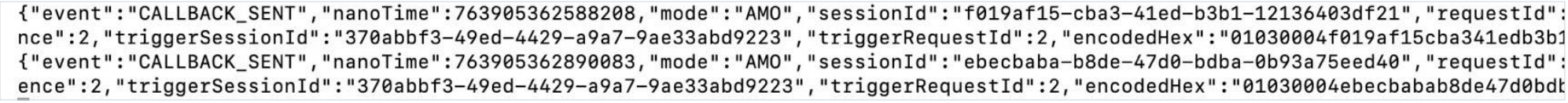


Figure 6a. Left-hand view of the two sequence-2 CALLBACK_SENT records: event, mode, registration identities and request IDs. Unmodified source screenshot 08, document viewport only.

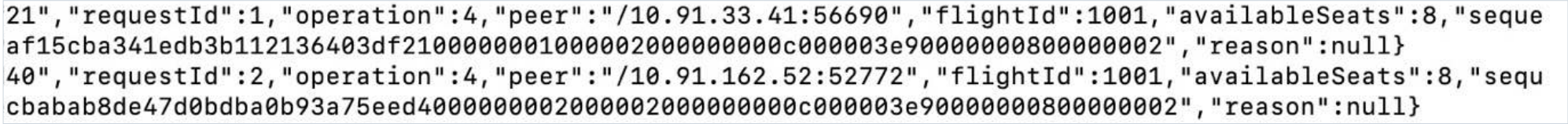


Figure 6b. Overlapping right-hand view of those same records: recipient endpoints, flight 1001, 8 seats and sequence 2. Source screenshot 08, document viewport only.

The recipients are C at 10.91.33.41:56690 and B at 10.91.162.52:52772. Both callbacks share writer trigger request 2. The full original capture, including the sequence-1 pair, is retained in the evidence archive. Its server folder is labelled 05-dual-02 while B's folder is 05-dual-en-01; the session and request identities above, not folder names, establish the correspondence.

Figure 7. Monitoring different flights

```
PS D:\ntu\103\groupwork> python -X utf8 -m client --server 10.91.127.7 --port 6789 --semantics amo --log-file evidence/d
emo/20261012-06-isolation-en-01/client-B.jsonl
SC6103 Flight Client; mode=AMO; session=2ce9ff0b-55f9-4f7e-8f9e-26c717a946c9

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 2
Flight ID: 1001
Request 2ce9ff0b-55f9-4f7e-8f9e-26c717a946c9:1; attempts: 1
Departure 2026-10-17 09:30 UTC+8; airfare 100.00 SGD; available seats 10

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 4
Flight ID: 1001
Monitor duration in seconds (1..3600): 60
Request 2ce9ff0b-55f9-4f7e-8f9e-26c717a946c9:2; attempts: 1
Monitor confirmed: flight 1001, remaining 59999 ms; menu input is paused during monitoring.
Seat update: flight 1001, available seats 9, sequence 1
Monitoring ended for flight 1001.
```

Figure 7a. B monitored flight 1001 and displayed its update to 9 seats, sequence 1. Original screenshot 28.

```
SC6103 Flight Client; mode=AMO; session=4d1cebe1-a338-41f5-b74a-d9f2b5f5e6b7

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 2
Flight ID: 1002
Request 4d1cebe1-a338-41f5-b74a-d9f2b5f5e6b7:1; attempts: 1
Departure 2026-10-17 14:00 UTC+8; airfare 180.00 SGD; available seats 20

1 Query route  2 Flight details  3 Reserve seats  4 Monitor seats  5 Set airfare  6 Increase airfare  0 Exit
Select an option: 4
Flight ID: 1002
Monitor duration in seconds (1..3600): 60
Request 4d1cebe1-a338-41f5-b74a-d9f2b5f5e6b7:2; attempts: 1
Monitor confirmed: flight 1002, remaining 59990 ms; menu input is paused during monitoring.
Seat update: flight 1002, available seats 19, sequence 1
Monitoring ended for flight 1002.
```

Figure 7b. C monitored flight 1002 and displayed its update to 19 seats, sequence 1. Original screenshot 09.

B registration: 2ce9ff0b-55f9-4f7e-8f9e-26c717a946c9:2. C registration: 4d1cebe1-a338-41f5-b74a-d9f2b5f5e6b7:2. Writer e0354c3f-fc66-4455-a485-b6382c68e90c reserved 1001 in request 1 and 1002 in request 2 (original screenshot 10). Each flight has its own update sequence.

6. Fault experiments and observed results

6.1 Controlled first-reply loss across computers

The team ran six separate LAN experiments: RESERVE, INCREASE and SET under ALO and AMO. Each run began with a newly started server, flight 1001 at 10 seats and fare 100, and one fresh experiment client process. The controlled write identity was UUID 11111111-1111-4111-8111-111111111111, request ID 2. Request 1 read the initial state and request 3 read the final state. C's terminal panels show two attempts for each target mutation; the paired A server captures show one DROP_REPLY and the execution/cache behaviour listed below. Counts exclude the two queries.

Operation	Mode	Attempts	Final seats / fare (SGD)	Executions / cache hits
Reserve 1 seat	ALO	2	8 / 100	2 / 0
Reserve 1 seat	AMO	2	9 / 100	1 / 1
Increase by 20	ALO	2	10 / 140	2 / 0
Increase by 20	AMO	2	10 / 120	1 / 1
Set fare to 120	ALO	2	10 / 120	2 / 0
Set fare to 120	AMO	2	10 / 120	1 / 1

The reservation experiment exposes a duplicate effect: ALO deducts two seats for one logical reservation while AMO deducts one. INCREASE similarly changes 100 to 140 in ALO, but only to 120 in AMO. SET finishes at 120 in both modes because assignment is idempotent. Its server logs are therefore essential: the ALO capture shows two executions of request 2, whereas AMO shows one execution followed by CACHE_HIT with businessExecuted=false.

A read-only audit of all six original A server JSON Lines independently confirms identical retry bytes and source endpoints, clean initial state, final state, target execution/cache counts and one dropped reply per run. The retained audit is evidence/demo/audit-20261013/lan-audit.json. C's original terminal screenshots supply client-side observations; raw C JSON Lines are not reconstructed from screenshots.

The AMO server trace orders BUSINESS_EXECUTED -> HISTORY_SAVED -> DROP_REPLY -> repeated REQUEST_RECEIVED -> CACHE_HIT -> REPLY_SENT. The reservation client receives the original cached result of 9 seats (Figures 8-11). The original server screenshots are 12/14 for RESERVE, 17/19 for INCREASE, and 21/23 for SET. Client counterparts are 13/15, 16/18, and 20/22. This is one controlled run per operation/mode, not a statistical performance study.

Figure 8. Lost reservation reply: ALO versus AMO

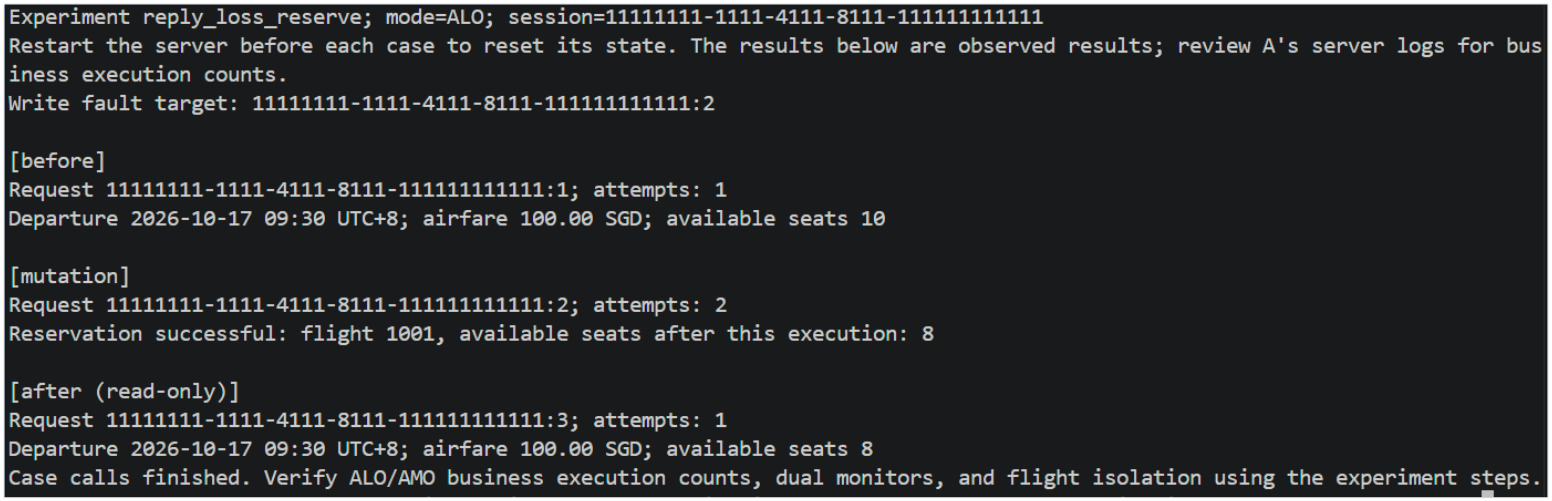


Figure 8a. ALO: initial 10 seats, two mutation attempts, final 8 seats. Original screenshot 13.

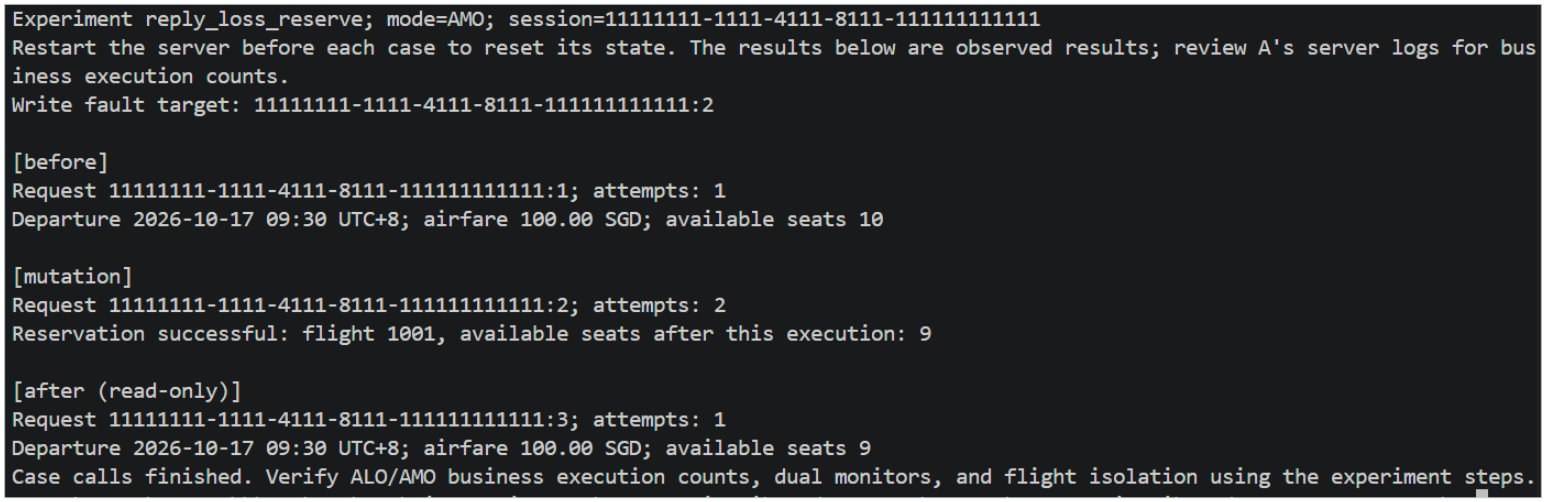


Figure 8b. AMO: the same initial state and two attempts, but final 9 seats. Original screenshot 15.

Server screenshots 12 and 14 show, respectively, two executions and one execution plus one cache hit for the same request identity. This demonstrates the duplicate effect on a non-idempotent reservation.

Figure 9. Lost increase-fare reply: ALO versus AMO

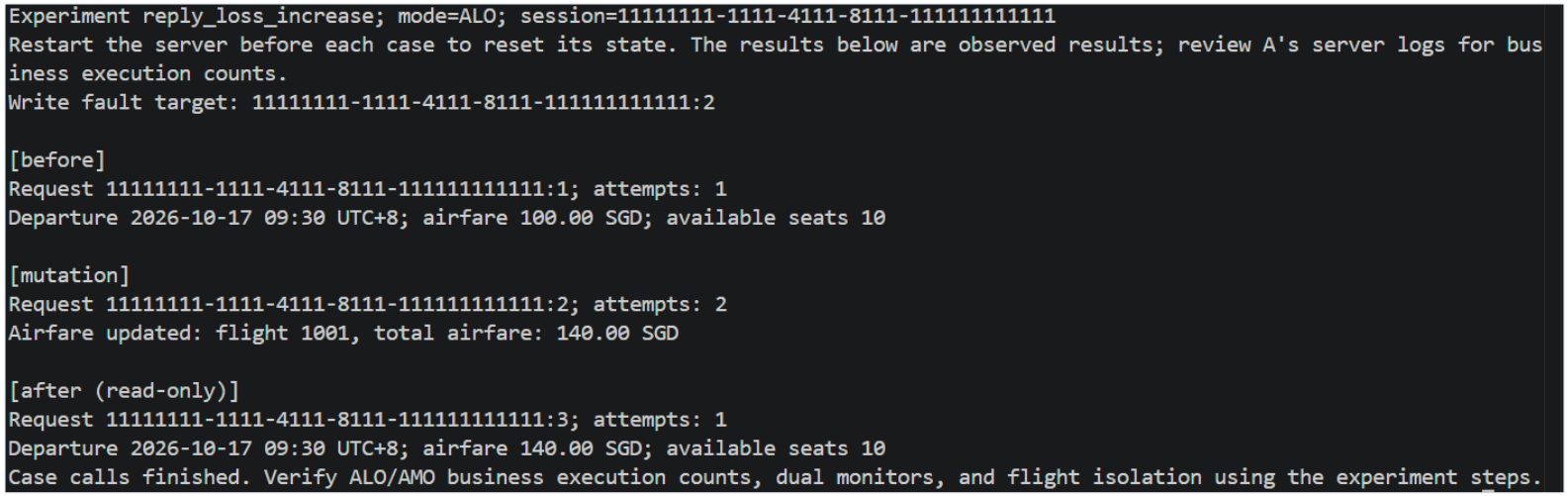


Figure 9a. ALO: adding 20 to an initial fare of 100 was applied twice, giving 140 SGD. Original screenshot 16.

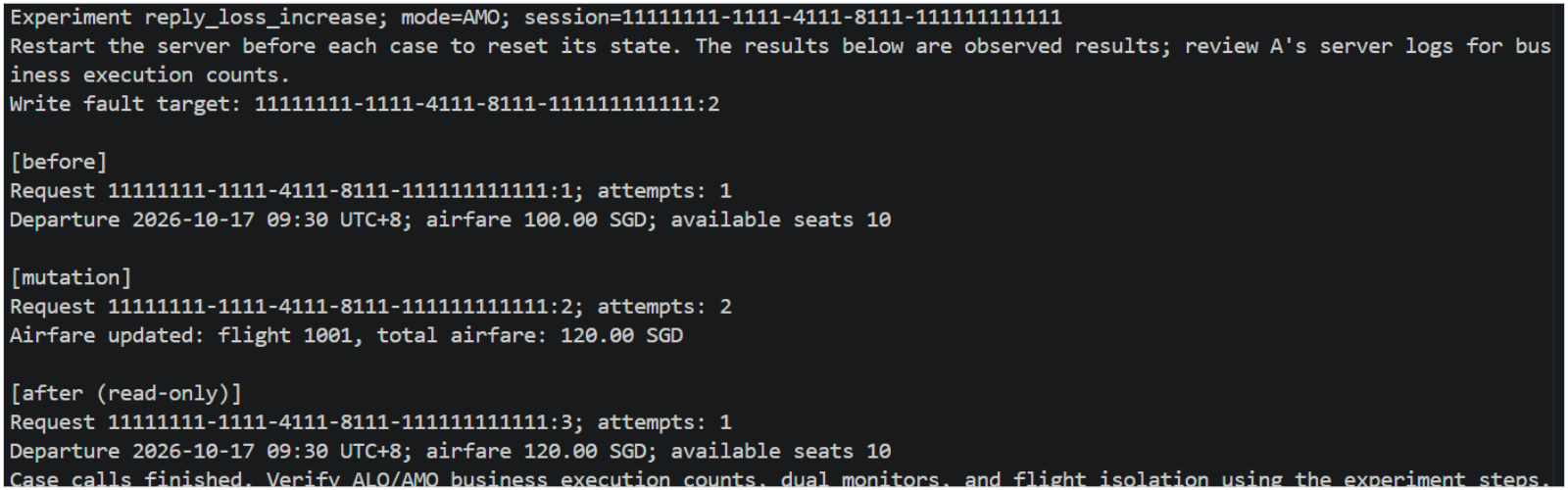


Figure 9b. AMO: two attempts produced only one addition, giving 120 SGD. Original screenshot 18.

The paired server captures 17 and 19 show the target operation 6 executing twice under ALO and being replayed from history under AMO. Seats remained 10 in both runs.

Figure 10. SET is idempotent, but execution counts differ

```
Experiment reply_loss_set; mode=ALO; session=11111111-1111-4111-8111-111111111111
Restart the server before each case to reset its state. The results below are observed results; review A's server logs for bus
iness execution counts.
Write fault target: 11111111-1111-4111-8111-111111111111:2

[before]
Request 11111111-1111-4111-8111-111111111111:1; attempts: 1
Departure 2026-10-17 09:30 UTC+8; airfare 100.00 SGD; available seats 10

[mutation]
Request 11111111-1111-4111-8111-111111111111:2; attempts: 2
Airfare updated: flight 1001, total airfare: 120.00 SGD

[after (read-only)]
Request 11111111-1111-4111-8111-111111111111:3; attempts: 1
Departure 2026-10-17 09:30 UTC+8; airfare 120.00 SGD; available seats 10
Case calls finished. Verify ALO/AMO business execution counts, dual monitors, and flight isolation using the experiment steps.
```

Figure 10a. ALO: two attempts ended at fare 120 SGD and 10 seats. Original screenshot 20.

```
Experiment reply_loss_set; mode=AMO; session=11111111-1111-4111-8111-111111111111
Restart the server before each case to reset its state. The results below are observed results; review A's server logs for bus
iness execution counts.
Write fault target: 11111111-1111-4111-8111-111111111111:2

[before]
Request 11111111-1111-4111-8111-111111111111:1; attempts: 1
Departure 2026-10-17 09:30 UTC+8; airfare 100.00 SGD; available seats 10

[mutation]
Request 11111111-1111-4111-8111-111111111111:2; attempts: 2
Airfare updated: flight 1001, total airfare: 120.00 SGD

[after (read-only)]
Request 11111111-1111-4111-8111-111111111111:3; attempts: 1
Departure 2026-10-17 09:30 UTC+8; airfare 120.00 SGD; available seats 10
Case calls finished. Verify ALO/AMO business execution counts, dual monitors, and flight isolation using the experiment steps.
```

Figure 10b. AMO: two attempts also ended at fare 120 SGD and 10 seats. Original screenshot 22.

Equal final fares do not prove duplicate suppression. Server screenshots 21 and 23 distinguish two SET executions in ALO from one execution and one cached replay in AMO.

Figure 11. AMO server cache evidence for the retried reservation



Figure 11a. Left-hand view of the dropped reply, repeated request and CACHE_HIT: request identity 2 and operation 3 are retained. Source screenshot 14, document viewport only.

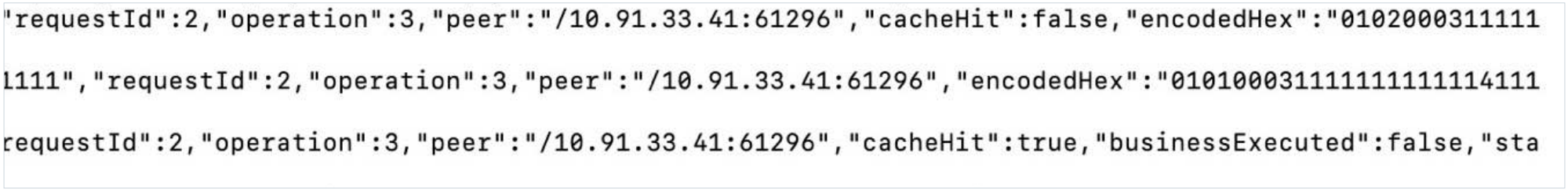


Figure 11b. Overlapping right-hand view of the same rows: the cached replay has cacheHit=true and businessExecuted=false. Source screenshot 14, document viewport only.

The original screenshot also shows HISTORY_SAVED before DROP_REPLY, and the final query at 9 seats. The viewport only changes document framing; the archived and embedded screenshot bytes are unchanged.

6.2 Additional local UDP loss matrix

The retained English-console verification contains sixteen local UDP runs with independent Java and Python processes on one computer. These extend the six LAN first-reply-loss observations with first-request loss, complete loss and quiet expiry. They are explicitly local loopback results. Each case resets the server and records client/server logs; complete-loss cases use the existing UDP proxy. E denotes target business executions and H denotes cache hits.

Case	Local ALO observation	Local AMO observation
Baseline, no loss	Seats 9, fare 140	Seats 9, fare 140
First reserve request lost	Seats 9; E=1, H=0	Seats 9; E=1, H=0
First reserve reply lost	Seats 8; E=2, H=0	Seats 9; E=1, H=1
First increase reply lost	Fare 140; E=2, H=0	Fare 120; E=1, H=1
First set-fare reply lost	Fare 120; E=2, H=0	Fare 120; E=1, H=1
All reserve requests lost	Seats 10; E=0, H=0; unknown	Seats 10; E=0, H=0; unknown
All reserve replies lost	Seats 5; E=5, H=0; unknown	Seats 9; E=1, H=4; unknown
Quiet monitor expiry	Completed at deadline	Completed at deadline

First-request loss is different from first-reply loss: the first request never reaches the server, so both modes execute the eventual successful retry once. Complete-loss runs exhaust all five attempts. The caller cannot distinguish 'nothing executed' from 'execution occurred but no reply arrived' purely from a timeout, which motivates reporting unknown instead of failure and avoiding a new-ID automatic write retry.

The latest matrix result file is evidence/a/submission-audit-20261013/matrix/results.csv, with all sixteen rows labelled PASSED_LOCAL_UDP; the earlier Windows matrix remains in evidence/c/en-20261012/. Per-case logs and server audits are retained. The analyzer correlates request identities and encoded bytes with execution/history/cache/drop events, rather than inferring server counts from client attempts. Its monitor regressions reject synthetic end timestamps before the recorded deadline.

7. Verification and experimental environment

7.1 Automated evidence

On 13 October, a clean source ZIP was extracted without Git metadata or compiled classes. All 200 tests and sixteen local UDP runs passed on macOS ARM64, Python 3.13.1 and actual Temurin Java 17.0.20.1; javac 21.0.5 compiled with --release 17. Original outputs and source hashes are in evidence/a/submission-audit-20261013/.

Verification suite	Passed tests
A binary protocol	19
A invocation semantics and transport	21
B business logic	34
B monitor service	15
C client	63
Experiment tools and evidence regressions	36
Real Java/Python boundary integration	12
Total automated tests	200
Additional local UDP runs	16

Historical Windows evidence in evidence/c/en-20261012/ records Windows 11, Python 3.13.3 and Java 17.0.13. Its verification notes distinguish reused suites from reruns. Earlier macOS evidence remains in evidence/a/release-java17/.

Integration covers invalid packets, mode mismatch, UTF-8, duplicate/session isolation, business-error replay, expiry and concurrent callbacks. Tests using injected send failures are distinguished from actual UDP observations.

7.2 Three-computer rehearsal on 12 October 2026

Role	Observed IPv4	Component / evidence
A - Zhang Zhiyin	10.91.127.7	Java server; macOS terminal captures; UDP port 6789
B - Peng Jinyu	10.91.162.52	Python query/monitor client; Windows PowerShell; retained B JSON Lines
C - Ji Chengyu	10.91.33.41	Python reservation/monitor client; peer address visible in A's captures

A's LAN environment records macOS ARM64 and Java 21.0.5. B/C exact LAN runtime versions were not captured; the historical Windows versions above are not attributed to both clients. Supported targets are Java 17+ and Python 3.10+.

A/B folder names differ; matching transaction identities link their logs. Screenshot 27 duplicates 07; screenshot 11 repeats the dual-monitor capture, not isolation. Original isolation server JSON Lines are now retained separately.

8. Reproduction, conclusions and limitations

Build from the repository root using sh server/build.sh on macOS/Linux or server/build.ps1 on Windows. Start a new server to reset flights, subscriptions and history. Both client and server must select the same semantics.

```
java -cp server/build/classes flight.ServerMain --bind 0.0.0.0 --port 6789 --semantics amo
python -X utf8 -m client --server <server IPv4> --port 6789 --semantics amo
```

For a controlled first-reply-loss run, restart the server with the selected mode and add --drop-first-reply 11111111-1111-4111-8111-111111111111:2. Start one new experiment process using the same mode and the fixed session. Its target mutation is request 2 after the initial query. For example:

```
python -X utf8 -m client --server <server IPv4> --semantics amo
--session-id 11111111-1111-4111-8111-111111111111
--case reply_loss_reserve --flight-id 1001 --quantity 1
```

The displayed wrapped command should be entered on one shell line. Use reply_loss_increase with --delta 20 or reply_loss_set with --new-price 120 for the fare cases. Do not reuse the fixed identity across cases without restarting the server. On macOS/Linux, python3 may replace python. The --check option is offline and does not establish network connectivity.

For concurrent monitoring, B and C each register 1001 for 60 seconds, then a third client process on C's computer reserves one seat twice. For flight isolation, B registers 1001 and C registers 1002; the writer reserves one seat on each. Wait for registration confirmation before writing and keep the writes within both active intervals. Verify natural expiry and correlate server events with the original registration identities.

The observed system implements the six services, cross-language binary encoding, server callbacks, bounded retries and selectable invocation semantics. The six LAN fault runs support the expected ALO/AMO distinction for both non-idempotent writes and idempotent SET. Concurrent-client and expiry observations corroborate the functional design. Results are controlled observations, not delivery probability or throughput measurements.

State and request history are held in memory; restarting the server discards both. History has no eviction during a process lifetime. Callbacks are best effort with no acknowledgement/retransmission. The system has no authentication or crash-persistent transaction record and makes no exactly-once or production-reliability claim. The LAN isolation run is supported by both the client observations and A's original per-recipient server delivery records.

The three-computer rehearsal documented here occurred on 12 October. The formal classroom demonstration is a separate future event: slot 11, 17 October 2026, 14:15-14:30 at LT8. No presentation slides are required.

One group representative must submit the report and one archive containing all well-commented source programs through the respective NTULearn items by 14 October 2026. Each member must separately submit the compulsory peer evaluation form covering themselves and both teammates. These submissions and the formal demo are not completed by publishing this repository.

9. Evidence index and references

All 28 supplied screenshots are preserved at evidence/demo/20261012-submitted-screenshots/originals/. manifest.json records their original order, labels, pixel dimensions and SHA-256 digests. There are 27 unique byte hashes. Eighteen selected originals are embedded or shown through document viewports; no terminal output has been regenerated or altered. Raw B logs and A server logs for the basic, dual and isolation sessions are retained in their original run folders. A server logs for all six LAN fault runs are also retained. The dated audit records 94 successful evidence checks and input-file hashes; it is an audit of the recorded rehearsal, not 94 additional application tests.

Experiment	Original supplied screenshot numbers	Supporting records
Queries and UTF-8	24	B basic session, requests 1-3
Business errors and fares	01-04	C basic session, errors 3-7 and fares 10-12
Single monitor and expiry	04, 25, 26	B basic registration 4; C reservations 8 and 13
Concurrent monitors	05-08; 11 and 27 are repeat views	B/C registrations; writer requests 1 and 2
Flight isolation	09, 10, 28	B/C observations and A server log 06-isolation-01; two correct recipients
Reserve reply loss	12-15	Target request 2, operation 3, ALO/AMO
Increase reply loss	16-19	Target request 2, operation 6, ALO/AMO
SET reply loss	20-23	Target request 2, operation 5, ALO/AMO

1. SC6103 Course.Project.for2026a.pdf, supplied four-page assignment: UDP services, invocation semantics, report and demonstration requirements.

2. Repository protocol specification v1.0, seed manifest and experiment cases: binary field layout, status definitions and fixed test data.

3. evidence/a/submission-audit-20261013/: clean-source audit, exact source hashes, runtime, 200 tests and sixteen-run matrix. Historical Windows evidence: evidence/c/en-20261012/.

4. evidence/a/release-java17/: historical clean-source Java 17 verification and boundary-integration evidence.

5. The supplied original screenshot archive, retained A/B JSON Lines and evidence/demo/audit-20261013/lan-audit.json described above.

6. Source repository: https://github.com/Stevenhh7/SC6103-Distributed-System-Coursework.

7. Instructor's submission/demo notice supplied by the team, and the demo signup sheet: https://docs.google.com/spreadsheets/d/1AiNb86SMPOP1NyYXTh8I7USKg1epzmb3PgcVq9U9JeM/edit. Slot 11 was verified on 13 October and confirmed by the team.